

Implémentation d'une pile

0.1 Définition et principe

Une *pile* est une structure de données LIFO (*Last In, First Out*) : le dernier élément ajouté est le premier retiré.

Les opérations usuelles doivent être en temps constant :

Opération	Complexité
empiler	$\Theta(1)$
depiler	$\Theta(1)$
estVide	$\Theta(1)$
estPleine	$\Theta(1)$ (pile bornée)

1 Implémentation naïve par maillons chaînés

Structure de la pile

```
typedef struct sMaillon {  
    int data;  
    struct sMaillon *suivant;  
} maillon;
```

Une pile vide est représentée par un pointeur NULL.

Tests d'état

```
bool estVide(maillon *p) {  
    return p == NULL;  
}
```

Empiler un élément

```
void empiler(maillon **p, int e) {  
    maillon *nouveau = malloc(sizeof(maillon));  
    nouveau->data = e;  
    nouveau->suivant = *p;  
    *p = nouveau;  
}
```

⚠ On utilise un double pointeur car on modifie à l'adresse du premier pointeur → ceci vient du choix de ne pas utiliser d'interface.

Dépiler un élément

```
int depiler(maillon **p) {
    assert(!estVide(*p));

    int val = (*p)->data;
    maillon *tmp = *p;
    *p = (*p)->suivant;
    free(tmp);

    return val;
}
```

Cette implémentation permet une pile de taille non bornée, au prix d'allocations dynamiques.

2 Implémentation d'une pile bornée par tableau statique

Structure de la pile

```
const int TAILLE = 10;

typedef struct {
    int tab[TAILLE];
    int sommet;
} pile;
```

Initialisation

```
void creerVideStat(pile *p) {
    p->sommet = -1;
}
```

Tests d'état

```
bool estVideStat(pile *p) {  
    return p->sommet == -1;  
}  
  
bool estPleineStat(pile *p) {  
    return p->sommet == TAILLE - 1;  
}
```

Empiler

```
void empilerStat(pile *p, int e) {  
    assert(!estPleineStat(p));  
    p->sommet++;  
    p->tab[p->sommet] = e;  
}
```

Dépiler

```
int depilerStat(pile *p) {  
    assert(!estVideStat(p));  
    int val = p->tab[p->sommet];  
    p->sommet--;  
    return val;  
}
```

Cette implémentation garantit :

- aucune allocation dynamique,
- des opérations en $\Theta(1)$,
- des invariants simples et robustes.